

Contents

Apuntes de Scala 2.12.21 — Guía para resolver la práctica	1
1. val y var	1
2. Tipos básicos e inferencia de tipos	2
3. Tipos numéricos: Double vs Float	2
4. Interpolación de strings	3
5. Funciones (def)	3
6. if / else if / else como expresión	4
7. Operadores relacionales y lógicos	4
8. Array (mutable en contenido, tamaño fijo)	5
9. List, Nil, :: y :::	5
10. Bucle while (estilo imperativo)	6
11. foreach (estilo funcional)	6
12. Efectos secundarios vs funciones puras	7
13. Mutabilidad vs inmutabilidad — resumen para justificar decisiones	8
14. Plantilla de proyecto sbt (para Partes 1.2, 1.3, 3.1 y 3.2)	8
15. Checklist rápido de conceptos por ejercicio	9
Ejercicios resueltos	9
Ejercicio 1 — Variables, tipos e inferencia	9
Ejercicio 2 — val, var y reasignación	10
Ejercicio 3 — Tipos numéricos y precisión	10
Ejercicio 4 — Función bust	11
Ejercicio 5 — Comparación de dos manos: maxHand	11
Ejercicio 6 — Decidir el ganador de una partida	12
Ejercicio 7 — Arrays y mutabilidad	12
Ejercicio 8 — Creación e inicialización de Arrays	12
Ejercicio 9 — Recorrer un Array con while	13
Ejercicio 10 — Listas e inmutabilidad	13
Ejercicio 11 — Construcción y concatenación de listas	13
Ejercicio 12 — Operadores relacionales y lógicos	14
Ejercicio 13 — foreach y funciones como valores	14
Ejercicio 14 — Efectos secundarios y estilo de programación	15
Ejercicio 15 — Programa integrado: torneo de Twenty-One	15
Mini proyecto 3.1 (VS Code) — Clasificador de resultados de un torneo de Twenty-One	16
Mini proyecto 3.2 (IntelliJ IDEA) — Analizador de calificaciones de un grupo	18

Apuntes de Scala 2.12.21 — Guía para resolver la práctica

Formato de cada apunte: **Concepto** → **Explicación** → **Sintaxis básica** → **Ejemplo práctico**. Todos los ejemplos usan solo elementos de los capítulos 1–3 (val/var, tipos, funciones, Array, List, if/else, operadores, while, foreach).

1. val y var

Explicación: val declara una referencia **inmutable**: una vez asignada no puede reasignarse (aunque, si el valor es un array, sus *elementos* sí pueden cambiar). var declara una referencia **mutable**, que puede reasignarse cuantas veces se quiera.

Sintaxis básica:

```
val nombreInmutable = valor
var nombreMutable = valor
```

Ejemplo práctico:

```
val jugador = "Marta"    // no se puede reasignar
var puntuacion = 10      // sí se puede reasignar

puntuacion = puntuacion + 5
puntuacion = puntuacion + 3
println(puntuacion)      // 18

jugador = "Ana"          // ERROR: reassignment to val
```

Al intentar reasignar jugador, Scala lanza un error de compilación del tipo: error: reassignment to val.

2. Tipos básicos e inferencia de tipos

Explicación: Scala tiene tipado estático, pero normalmente no hace falta escribir el tipo: el compilador lo **infiere** a partir del valor asignado. Declarar el tipo explícitamente sirve para dejar clara la intención o cuando el valor inicial es ambiguo.

Tipos básicos más usados: Int, Double, Float, Boolean, String.

Sintaxis básica:

```
val nombre: String = "Scala"    // tipo explícito
val version = "2.12.21"         // tipo inferido (String)
```

Ejemplo práctico:

```
// Versión A: tipo explícito
val nombreA: String = "Marta"
val partidasA: Int = 12
val puntuacionA: Double = 87.5
val porcentajeVictoriasA: Double = 0.65
val activoA: Boolean = true

// Versión B: tipo inferido
val nombreB = "Marta"
val partidasB = 12
val puntuacionB = 87.5
val porcentajeVictoriasB = 0.65
val activoB = true
```

Scala infiere String, Int, Double y Boolean exactamente igual que si se hubieran escrito a mano; la diferencia es solo de legibilidad, no de comportamiento.

3. Tipos numéricos: Double vs Float

Explicación: Double tiene 64 bits de precisión (más decimales exactos), Float tiene 32 bits (menos precisión, puede redondear). Para contar cosas (nº de estudiantes) se usa Int; para valores con decimales de alta precisión, Double.

Sintaxis básica:

```
val d: Double = 3.14159265358979323846
val f: Float = 3.14159265358979323846f    // nótese la 'f' final
```

Ejemplo práctico:

```
val piDouble: Double = 3.14159265358979323846264338327
val piFloat: Float = 3.14159265358979323846264338327f

println(piDouble) // 3.141592653589793 (más decimales conservados)
println(piFloat)  // 3.1415927 (menos precisión, se pierde exactitud)

val numeroEstudiantes: Int = 30
val haAprobado: Boolean = true
val cursoNombre: String = "Programación con Scala"
```

4. Interpolación de strings

Explicación: El prefijo `s` delante de una cadena permite insertar variables directamente con `$variable` o expresiones con `${...}`.

Sintaxis básica:

```
println(s"Texto con $variable y ${expresion + 1}")
```

Ejemplo práctico:

```
val nombre = "Scala"
val version = "2.12.21"
println(s"Hola desde $nombre $version")
// Hola desde Scala 2.12.21
```

5. Funciones (def)

Explicación: Una función se declara con `def`, seguida del nombre, los parámetros con su tipo, y (opcionalmente) el tipo de retorno. El valor de la **última expresión** del cuerpo es el que se devuelve (no hace falta `return`).

Sintaxis básica:

```
def nombreFuncion(parametro1: Tipo1, parametro2: Tipo2): TipoRetorno = {
  // cuerpo
  ultimaExpresion
}
```

Ejemplo práctico:

```
def bust(puntuacion: Int): Boolean = {
  puntuacion > 21
}

println(bust(18)) // false
println(bust(21)) // false
println(bust(22)) // true
println(bust(30)) // true
```

Una función con dos parámetros y `if/else`:

```
def maxHand(a: Int, b: Int): Int = {
  if (a > b) a
```

```

    else b
  }

println(maxHand(17, 19)) // 19
println(maxHand(21, 21)) // 21 (empate: devuelve cualquiera de los dos, son iguales)

```

6. if / else if / else como expresión

Explicación: En Scala, if/else **devuelve un valor**, por lo que se puede usar directamente como cuerpo de una función o para asignar una variable, encadenando condiciones con else if.

Sintaxis básica:

```

if (condicion1) valor1
else if (condicion2) valor2
else valorPorDefecto

```

Ejemplo práctico:

```

def ganador(handA: Int, handB: Int): Int = {
  if (bust(handA) && bust(handB)) 0
  else if (bust(handA)) handB
  else if (bust(handB)) handA
  else maxHand(handA, handB)
}

println(ganador(26, 20)) // 20
println(ganador(18, 22)) // 18
println(ganador(24, 25)) // 0
println(ganador(17, 19)) // 19
println(ganador(21, 20)) // 21

```

7. Operadores relacionales y lógicos

Explicación: Comparan valores (relacionales) o combinan condiciones booleanas (lógicos).

Tipo	Operadores
Relacionales	> < >= <= == !=
Lógicos	&& (Y), \ \ (O), ! (NO)

Sintaxis básica:

```

valor1 > valor2
condicionA && condicionB
!condicion

```

Ejemplo práctico:

```

val handA = 18
val handB = 21
val handC = 25

println(handA > handB) // false

```

```
println(handB == 21)           // true
println(handC != 21)          // true
println(handA <= 21 && handB <= 21) // true
println(handA > 21 || handC > 21) // true
println(!(handB > 21))        // true (handB no se ha pasado)
```

8. Array (mutable en contenido, tamaño fijo)

Explicación: Un Array es una colección de tamaño **fijo** cuyos elementos **sí pueden modificarse** (aunque esté declarado con val), porque val fija la referencia al array, no su contenido. No admite tipos distintos al declarado (Array[String] no acepta Int).

Sintaxis básica:

```
val arr = Array(elem1, elem2, elem3) // creación con valores
val arr2 = new Array[Int](n)         // creación vacía de tamaño n
arr(indice) = nuevoValor             // modificar un elemento
arr.length                           // longitud
```

Ejemplo práctico:

```
val jugadores = Array("Alex", "Chen", "Marta")
println(jugadores.mkString(", ")) // Alex, Chen, Marta

jugadores(0) = "Sindhu"           // se puede modificar el contenido
println(jugadores.mkString(", ")) // Sindhu, Chen, Marta

// jugadores(0) = 500              // ERROR: type mismatch, se esperaba String

val puntuaciones = new Array[Int](4) // [0, 0, 0, 0] valores por defecto
puntuaciones(0) = 17
puntuaciones(1) = 24
puntuaciones(2) = 21
puntuaciones(3) = 19
println(puntuaciones.mkString(", ")) // 17, 24, 21, 19
println(puntuaciones.length)         // 4
```

Un Array[Int] recién creado con new Array[Int](n) se inicializa con ceros (0) en todas sus posiciones.

9. List, Nil, :: y :::

Explicación: Una List es **inmutable**: cualquier operación que “añade” elementos en realidad crea una **lista nueva**, sin modificar la original. Nil es la lista vacía. :: añade un elemento al principio (construye una lista nueva). ::: concatena dos listas (también genera una lista nueva).

Sintaxis básica:

```
val lista = List(a, b, c)
val listaConstruida = a :: b :: c :: Nil
val nuevaLista = elemento :: listaExistente
val concatenada = lista1 ::: lista2
lista.reverse
lista.length
```

Ejemplo práctico:

```
val jugadores = List("Alex", "Chen", "Marta")
val conNuevo = "Sindhu" :: jugadores

println(jugadores)    // List(Alex, Chen, Marta)    <- sin cambios
println(conNuevo)     // List(Sindhu, Alex, Chen, Marta)
println(jugadores.length) // 3
println(jugadores.reverse) // List(Marta, Chen, Alex)

// Construcción con Nil y ::
val listaA = "Ana" :: "Luis" :: "Marta" :: Nil
val listaB = "Pedro" :: "Sofia" :: Nil
val resultado = listaA ::: listaB

println(listaA)        // List(Ana, Luis, Marta)    <- sin cambios
println(listaB)        // List(Pedro, Sofia)        <- sin cambios
println(resultado)     // List(Ana, Luis, Marta, Pedro, Sofia)
```

:: y ::: no modifican las listas originales porque List es inmutable: cada operación devuelve una estructura nueva que reutiliza internamente los nodos de la original.

10. Bucle while (estilo imperativo)

Explicación: Recorre mientras una condición sea verdadera. Necesita un **contador mutable** (var) que se debe incrementar manualmente dentro del bucle; si no se incrementa, el bucle nunca termina (bucle infinito).

Sintaxis básica:

```
var i = 0
while (i < coleccion.length) {
    // usar coleccion(i)
    i = i + 1
}
```

Ejemplo práctico:

```
val manos = Array(17, 24, 21, 19, 26)
var i = 0
while (i < manos.length) {
    println(s"Mano: ${manos(i)} -> Bust: ${bust(manos(i))}")
    i = i + 1
}
```

El contador i debe ser var porque se reasigna en cada vuelta; es el elemento más claramente **imperativo** del ejercicio (control manual de estado).

11. foreach (estilo funcional)

Explicación: Recorre una colección aplicando una función a cada elemento, **sin necesidad de contador ni variable mutable**. Se acerca más al estilo funcional porque describe *qué* hacer con cada elemento, no *cómo* iterar.

Sintaxis básica:

```
coleccion.foreach(elemento => accion(elemento))
// o con función ya definida:
coleccion.foreach(nombreFuncion)
```

Ejemplo práctico:

```
val manos = Array(17, 24, 21, 26, 18)

// Versión con while (imperativa)
var j = 0
while (j < manos.length) {
    println(bust(manos(j)))
    j = j + 1
}

// Versión con foreach (funcional)
manos.foreach(mano => println(bust(mano)))
```

Comparación: la versión while necesita un contador (j) y una variable mutable para recorrer; la versión foreach no necesita ninguna de las dos y se aproxima más al estilo funcional.

12. Efectos secundarios vs funciones puras

Explicación: Una función tiene **efecto secundario** cuando modifica algo fuera de su propio ámbito (por ejemplo, una variable var externa) en lugar de limitarse a devolver un resultado calculado a partir de sus parámetros. El estilo funcional prefiere funciones **puras**: reciben datos, devuelven un resultado nuevo, y no tocan nada externo.

Sintaxis básica:

```
// Con efecto secundario
var estadoExterno = 0
def modificaEstado(x: Int): Unit = {
    estadoExterno = estadoExterno + x
}

// Pura (sin efecto secundario)
def calcula(a: Int, b: Int): Int = a + b
```

Ejemplo práctico:

```
var total = 0
def sumarAlTotal(valor: Int) = {
    total = total + valor    // modifica una variable externa: efecto secundario
}

sumarAlTotal(5)
println(total)    // 5
sumarAlTotal(3)
println(total)    // 8

def sumar(a: Int, b: Int): Int = {
    a + b            // no modifica nada externo, solo devuelve un resultado
}
println(sumar(5, 3))    // 8, total no cambia
```

Característica	sumarAlTotal	sumar
Modifica datos externos	Sí	No
Devuelve un resultado calculado	No (Unit)	Sí
Utiliza efecto secundario	Sí	No
Estilo predominante	Imperativo	Funcional

13. Mutabilidad vs inmutabilidad — resumen para justificar decisiones

Explicación: En los informes se pide indicar qué partes de la solución son mutables/inmutables.
Regla práctica:

- **Immutable (val, List):** datos que no deberían cambiar tras crearse (nombres de jugadores, listas de partida).
- **Mutable (var, Array):** contadores de bucles (var i), puntuaciones que se van fijando en un Array.

Sintaxis básica:

```
val datosFijos = List(...) // immutable
var contador = 0           // mutable
val coleccionModificable = Array(...) // referencia fija, contenido mutable
```

Ejemplo práctico (aplicable al ejercicio integrador):

```
val jugadores = List("Alex", "Chen", "Marta", "Sindhu") // immutable: no cambia
val manos = Array(18, 24, 21, 20)                       // mutable: puede reescribirse

var i = 0 // mutable: contador del while
var mejor = -1 // mutable: se va actualizando al recorrer

while (i < manos.length) {
  if (!bust(manos(i)) && manos(i) > mejor) {
    mejor = manos(i)
  }
  i = i + 1
}
println(s"Mejor puntuación válida: $mejor")
```

- Mutable: manos (Array), i y mejor (contadores/acumuladores con var).
- Immutable: jugadores (List), y las funciones bust, maxHand, etc. (no modifican nada externo).
- Estilo funcional: uso de funciones puras como bust y de foreach.
- Estilo imperativo: el bucle while con contador i.

14. Plantilla de proyecto sbt (para Partes 1.2, 1.3, 3.1 y 3.2)

Explicación: Un proyecto sbt mínimo necesita un build.sbt en la raíz y el código fuente dentro de src/main/scala/. sbt compile compila; sbt run ejecuta la clase App.

Sintaxis básica:

```
proyecto/
├─ build.sbt
├─ project/
└─ src/
```



```
└─ main/
  └─ scala/
    └─ Main.scala
```

```
// build.sbt
scalaVersion := "2.12.21"
name := "nombre-proyecto"
```

```
// Main.scala
object Main extends App {
  println("Hola")
}
```

Ejemplo práctico (comandos de terminal):

```
java -version
javac -version
sbt --version
sbt compile
sbt run
```

15. Checklist rápido de conceptos por ejercicio

Ejercicio(s)	Conceptos clave a aplicar
1-3	val/var, tipos, inferencia, Double vs Float
4-6	funciones, if/else if/else, funciones puras
7-8	Array, mutabilidad de contenido, new Array[Int]
9	while, contador var, .length
10-11	List, Nil, ::, :::, inmutabilidad
12	operadores relacionales y lógicos
13	foreach vs while
14	efectos secundarios vs funciones puras
15 / mini proyectos	integración: List + Array + funciones + while/foreach + if/else + mutabilidad/inmutabilidad

Ejercicios resueltos

A continuación se resuelven de forma completa los 15 ejercicios de la Parte 2 (JupyterLab + Almond + Scala 2.12.21) y los dos mini proyectos de la Parte 3 (VS Code y IntelliJ IDEA). Todo el código usa únicamente los elementos vistos en los apuntes anteriores.

Ejercicio 1 — Variables, tipos e inferencia

```
// Versión A: tipo explícito
val nombreA: String = "Marta"
val partidasA: Int = 24
val puntuacionA: Double = 187.5
val porcentajeVictoriasA: Double = 0.62
```

```

val activoA: Boolean = true

// Versión B: tipo inferido
val nombreB = "Marta"
val partidasB = 24
val puntuacionB = 187.5
val porcentajeVictoriasB = 0.62
val activoB = true

println(s"$nombreA - partidas: $partidasA - puntuación: $puntuacionA - % victorias: $porcentajeVictoriasA")
println(s"$nombreB - partidas: $partidasB - puntuación: $puntuacionB - % victorias: $porcentajeVictoriasB")

```

Salida:

```

Marta - partidas: 24 - puntuación: 187.5 - % victorias: 0.62 - activo: true
Marta - partidas: 24 - puntuación: 187.5 - % victorias: 0.62 - activo: true

```

Documentación: Scala infiere String para nombreB, Int para partidasB, Double para puntuacionB y porcentajeVictoriasB, y Boolean para activoB; son los mismos tipos que se escribieron a mano en la Versión A. No hay diferencia de comportamiento, solo de legibilidad. nombre debería ser val (no cambia durante la partida); partidas, puntuacion y porcentajeVictorias deberían ser var, porque se actualizan tras cada partida; activo también podría ser var si el jugador puede abandonar y volver.

Ejercicio 2 — val, var y reasignación

```

val jugador = "Marta"
var puntuacion = 10

puntuacion = puntuacion + 5
puntuacion = puntuacion + 3
println(puntuacion) // 18

jugador = "Ana" // ERROR: reassignment to val

```

Salida:

```

18
error: reassignment to val
  jugador = "Ana"
    ^

```

Documentación: puntuacion puede modificarse porque está declarada con var, que permite reasignación. jugador no puede reasignarse porque está declarada con val, una referencia inmutable: una vez fijado su valor, no puede cambiarse. Scala genera el error reassignment to val al intentar la reasignación.

Ejercicio 3 — Tipos numéricos y precisión

```

val valorDouble: Double = 3.14159265358979323846264338327
val valorFloat: Float = 3.14159265358979323846264338327f

println(valorDouble) // 3.141592653589793

```

```
println(valorFloat)    // 3.1415927

val numeroEstudiantes: Int = 28
val haAprobado: Boolean = true
val nombreAsignatura: String = "Programación con Scala"

println(numeroEstudiantes)
println(haAprobado)
println(nombreAsignatura)
```

Documentación: Double conserva muchos más decimales exactos que Float, que pierde precisión a partir de la séptima u octava cifra. Para el número de estudiantes de una clase se usaría Int (es una cantidad entera). Para si un estudiante ha aprobado se usaría Boolean (solo hay dos estados posibles).

Ejercicio 4 — Función bust

```
def bust(puntuacion: Int): Boolean = {
  puntuacion > 21
}

println(bust(18))    // false
println(bust(21))    // false
println(bust(22))    // true
println(bust(30))    // true
```

Salida:

```
false
false
true
true
```

La función solo devuelve el resultado de la comparación `puntuacion > 21`; no modifica ninguna variable externa.

Ejercicio 5 — Comparación de dos manos: maxHand

```
def maxHand(a: Int, b: Int): Int = {
  if (a > b) a
  else b
}

println(maxHand(17, 19))    // 19
println(maxHand(20, 18))    // 20
println(maxHand(21, 21))    // 21
```

Documentación: Cuando ambas manos tienen el mismo valor, `a > b` es false, así que la función entra en la rama `else` y devuelve `b`. Como `a == b`, da igual devolver uno u otro: el resultado numérico es correcto en cualquier caso, así que no es necesario modificar la función para ese caso concreto.

Ejercicio 6 — Decidir el ganador de una partida

```
def ganador(handA: Int, handB: Int): Int = {  
  if (bust(handA) && bust(handB)) 0  
  else if (bust(handA)) handB  
  else if (bust(handB)) handA  
  else maxHand(handA, handB)  
}  
  
println(ganador(26, 20)) // 20  
println(ganador(18, 22)) // 18  
println(ganador(24, 25)) // 0  
println(ganador(17, 19)) // 19  
println(ganador(21, 20)) // 21
```

Documentación de cada prueba: - ganador(26, 20) → bust(26) es true y bust(20) es false → se cumple la segunda condición (bust(handA)) → devuelve handB = 20. - ganador(18, 22) → bust(18) es false, bust(22) es true → se cumple la tercera condición (bust(handB)) → devuelve handA = 18. - ganador(24, 25) → ambas superan 21 → se cumple la primera condición → devuelve 0. - ganador(17, 19) → ninguna se pasa → se cumple la rama else → maxHand(17, 19) = 19. - ganador(21, 20) → ninguna se pasa → maxHand(21, 20) = 21.

Ejercicio 7 — Arrays y mutabilidad

```
val jugadores = Array("Alex", "Chen", "Marta")  
println(jugadores.mkString(", ")) // Alex, Chen, Marta  
  
jugadores(0) = "Sindhu"  
println(jugadores.mkString(", ")) // Sindhu, Chen, Marta  
  
// jugadores(0) = 500 // ERROR: type mismatch; found: Int(500), required: String
```

Documentación: - Los elementos del array pueden cambiar aunque se haya declarado con val porque val fija la **referencia** al array (el array sigue siendo el mismo objeto en memoria), pero no impide modificar su contenido interno. - Scala no permite jugadores(0) = 500 porque el array se declaró como Array[String]; el compilador comprueba en tiempo de compilación que cada elemento sea del tipo declarado, y 500 es un Int. - Reasignar la variable sería jugadores = Array(...) (prohibido por ser val); modificar un elemento es jugadores(i) = valor (permitido, porque no cambia la referencia, solo el contenido).

Ejercicio 8 — Creación e inicialización de Arrays

```
val puntuaciones = new Array[Int](4)  
println(puntuaciones.mkString(", ")) // 0, 0, 0, 0  
  
puntuaciones(0) = 17  
puntuaciones(1) = 24  
puntuaciones(2) = 21  
puntuaciones(3) = 19  
  
println(puntuaciones.mkString(", ")) // 17, 24, 21, 19
```

```
println(puntuaciones.length)           // 4
```

Respuesta: Un `Array[Int]` recién creado con `new Array[Int](n)` contiene ceros (0) en todas sus posiciones antes de asignar manualmente sus elementos.

Ejercicio 9 — Recorrer un Array con while

```
val manos = Array(17, 24, 21, 19, 26)
var i = 0
while (i < manos.length) {
  println(s"Mano: ${manos(i)} -> Bust: ${bust(manos(i))}")
  i = i + 1
}
```

Salida:

```
Mano: 17 -> Bust: false
Mano: 24 -> Bust: true
Mano: 21 -> Bust: false
Mano: 19 -> Bust: false
Mano: 26 -> Bust: true
```

Documentación: El contador `i` debe ser `var` porque se reasigna en cada vuelta del bucle (`i = i + 1`); una `val` no admitiría esa reasignación. Si no se incrementara `i`, la condición `i < manos.length` sería siempre verdadera y el bucle no terminaría nunca (bucle infinito). El contador mutable, la condición de parada manual y el incremento explícito son los elementos propios del estilo imperativo de este ejercicio.

Ejercicio 10 — Listas e inmutabilidad

```
val jugadores = List("Alex", "Chen", "Marta")
val conSindhu = "Sindhu" :: jugadores

println(jugadores)           // List(Alex, Chen, Marta)
println(conSindhu)           // List(Sindhu, Alex, Chen, Marta)
println(jugadores)           // List(Alex, Chen, Marta) <- sigue sin cambios
println(jugadores.length)    // 3
println(jugadores.reverse)   // List(Marta, Chen, Alex)
```

Documentación: `::` no modifica `jugadores`; construye una **lista nueva** que tiene "Sindhu" como cabeza y reutiliza internamente los nodos de `jugadores` como cola. Como `List` es inmutable, ninguna operación puede alterar la lista original: solo se pueden crear listas nuevas a partir de ella.

Ejercicio 11 — Construcción y concatenación de listas

```
val listaA = "Ana" :: "Luis" :: "Marta" :: Nil
val listaB = "Pedro" :: "Sofia" :: Nil
val resultado = listaA ::: listaB
```

```
println(listaA)      // List(Ana, Luis, Marta)
println(listaB)      // List(Pedro, Sofia)
println(resultado)   // List(Ana, Luis, Marta, Pedro, Sofia)
```

Ambas listas originales se muestran sin cambios tras la concatenación, demostrando que `:::` genera una lista nueva.

Ejercicio 12 — Operadores relacionales y lógicos

```
val handA = 18
val handB = 21
val handC = 25

println(handA > handB)      // false
println(handB == 21)       // true
println(handC != 21)       // true
println(handA <= 21 && handB <= 21) // true
println(handA > 21 || handC > 21) // true
println(!(handB > 21))     // true
```

Expresión	Resultado	Explicación
handA > handB	false	18 no es mayor que 21
handB == 21	true	handB vale exactamente 21
handC != 21	true	25 es distinto de 21
handA <= 21 && handB <= 21	true	ambas condiciones se cumplen a la vez
handA > 21 handC > 21	true	basta con que una se cumpla (handC sí supera 21)
!(handB > 21)	true	handB > 21 es false, y !false es true

Ejercicio 13 — foreach y funciones como valores

```
val manos = Array(17, 24, 21, 26, 18)

// Versión A: while
var j = 0
while (j < manos.length) {
  println(bust(manos(j)))
  j = j + 1
}

// Versión B: foreach
manos.foreach(mano => println(bust(mano)))
```

Salida (ambas versiones):

```
false
true
false
```

true
false

Documentación: La versión while necesita un contador (j) para saber en qué posición está y una variable mutable (var j) para poder avanzar. La versión foreach no necesita ni contador ni variable mutable: se limita a describir qué hacer con cada elemento. Por eso foreach se aproxima más al estilo funcional presentado en el material del curso.

Ejercicio 14 — Efectos secundarios y estilo de programación

```
var total = 0

def sumarAlTotal(valor: Int) = {
  total = total + valor
}

sumarAlTotal(5)
println(total)    // 5
sumarAlTotal(10)
println(total)    // 15
sumarAlTotal(3)
println(total)    // 18

def sumar(a: Int, b: Int): Int = {
  a + b
}

println(sumar(5, 10))    // 15, total sigue en 18
println(total)           // 18
```

Documentación: sumarAlTotal modifica total, una variable declarada **fuera** de su propio cuerpo, en lugar de limitarse a devolver un resultado; ese es precisamente el efecto secundario. sumar solo usa sus parámetros y devuelve un valor nuevo, sin tocar nada externo.

Característica	sumarAlTotal	sumar
Modifica datos externos	Sí	No
Devuelve un resultado calculado	No (Unit)	Sí
Utiliza efecto secundario	Sí	No
Estilo predominante	Imperativo	Funcional

Ejercicio 15 — Programa integrado: torneo de Twenty-One

```
val jugadores = List("Alex", "Chen", "Marta", "Sindhu")
val manos = Array(18, 24, 21, 20)

def procesarRonda(manos: Array[Int], jugadores: List[String]): Unit = {
  var mejor = -1
  var i = 0
  while (i < manos.length) {
    val jugador = jugadores(i)
```

```

    val puntuacion = manos(i)
    if (bust(puntuacion)) {
        println(s"$jugador -> $puntuacion -> BUST")
    } else {
        println(s"$jugador -> $puntuacion -> VALIDA")
        if (puntuacion > mejor) mejor = puntuacion
    }
    i = i + 1
}
println(s"Mejor puntuación válida: $mejor")
}

println("--- Ronda 1 ---")
procesarRonda(manos, jugadores)

val manosRonda2 = Array(22, 19, 20, 21)
println("--- Ronda 2 ---")
procesarRonda(manosRonda2, jugadores)

```

Salida:

```

--- Ronda 1 ---
Alex -> 18 -> VALIDA
Chen -> 24 -> BUST
Marta -> 21 -> VALIDA
Sindhu -> 20 -> VALIDA
Mejor puntuación válida: 21
--- Ronda 2 ---
Alex -> 22 -> BUST
Chen -> 19 -> VALIDA
Marta -> 20 -> VALIDA
Sindhu -> 21 -> VALIDA
Mejor puntuación válida: 21

```

Informe final: - **Mutables:** manos y manosRonda2 (Array), y dentro de procesarRonda: mejor y i (contadores/acumuladores con var). - **Inmutables:** jugadores (List), que no se modifica en ninguna ronda. - **Funciones:** bust (reutilizada del ejercicio 4) y procesarRonda (definida para no repetir código entre rondas). - **Estructuras de control:** while para recorrer las puntuaciones, if/else para clasificar cada mano. - **Más cercano al estilo funcional:** el uso de bust como función pura que no modifica nada externo. - **Más cercano al estilo imperativo:** el bucle while con el contador i y el acumulador mejor, que se van actualizando paso a paso.

Mini proyecto 3.1 (VS Code) — Clasificador de resultados de un torneo de Twenty-One

build.sbt:

```

scalaVersion := "2.12.21"
name := "torneo-twenty-one"

```

src/main/scala/Main.scala:

```

object Main extends App {

    def bust(puntuacion: Int): Boolean = puntuacion > 21

```



```

def estadoMano(puntuacion: Int): String = {
  if (bust(puntuacion)) "BUST" else "VALIDA"
}

def mejorMano(handA: Int, handB: Int): Int = {
  if (bust(handA) && bust(handB)) 0
  else if (bust(handA)) handB
  else if (bust(handB)) handA
  else if (handA > handB) handA
  else handB
}

val jugadores = List("Alex", "Chen", "Marta", "Sindhu", "Luis")

def procesarRonda(nombreRonda: String, puntuaciones: Array[Int]): Int = {
  println(s"--- $nombreRonda ---")

  var i = 0
  var validas = 0
  var busts = 0
  var mejor = -1

  while (i < puntuaciones.length) {
    val jugador = jugadores(i)
    val puntuacion = puntuaciones(i)
    println(s"$jugador -> $puntuacion -> ${estadoMano(puntuacion)}")

    if (bust(puntuacion)) {
      busts = busts + 1
    } else {
      validas = validas + 1
      if (puntuacion > mejor) mejor = puntuacion
    }
    i = i + 1
  }

  println("--- Resumen de la ronda ---")
  println(s"Jugadores: ${puntuaciones.length}")
  println(s"Manos válidas: $validas")
  println(s"Bust: $busts")
  println(s"Mejor puntuación válida: $mejor")
  mejor
}

val puntuacionesRonda1 = Array(18, 24, 21, 20, 26)
val puntuacionesRonda2 = Array(22, 19, 20, 21, 17)

val mejorRonda1 = procesarRonda("Ronda 1", puntuacionesRonda1)
val mejorRonda2 = procesarRonda("Ronda 2", puntuacionesRonda2)

println("--- Comparación de rondas ---")
println(s"Mejor puntuación ronda 1: $mejorRonda1")
println(s"Mejor puntuación ronda 2: $mejorRonda2")

```

```

if (mejorRonda1 > mejorRonda2) {
  println("La ronda 1 tuvo la mejor puntuación")
} else if (mejorRonda2 > mejorRonda1) {
  println("La ronda 2 tuvo la mejor puntuación")
} else {
  println("Ambas rondas tuvieron la misma mejor puntuación")
}

// Versión con foreach, para comparar con el while anterior
println("--- Estado de cada mano (foreach) ---")
puntuacionesRonda1.foreach(p => println(s"$p -> ${estadoMano(p)}"))
}

```

Salida esperada (resumen):

```

--- Ronda 1 ---
Alex -> 18 -> VALIDA
Chen -> 24 -> BUST
Marta -> 21 -> VALIDA
Sindhu -> 20 -> VALIDA
Luis -> 26 -> BUST
--- Resumen de la ronda ---
Jugadores: 5
Manos válidas: 3
Bust: 2
Mejor puntuación válida: 21
...
--- Comparación de rondas ---
Mejor puntuación ronda 1: 21
Mejor puntuación ronda 2: 21
Ambas rondas tuvieron la misma mejor puntuación

```

Comparación while vs foreach (para el README del mini proyecto): la versión while necesita el contador `i` (una variable mutable) para recorrer el array y saber cuándo parar; la versión `foreach` no necesita contador ni variable mutable, y se limita a aplicar `estadoMano` a cada elemento — es la que más se acerca al estilo funcional.

Mini proyecto 3.2 (IntelliJ IDEA) — Analizador de calificaciones de un grupo

build.sbt:

```

scalaVersion := "2.12.21"
name := "analizador-notas"

```

src/main/scala/Main.scala:

```

object Main extends App {

  def aprobado(nota: Int): Boolean = nota >= 5

  def estadoNota(nota: Int): String = {
    if (aprobado(nota)) "APROBADO" else "SUSPENSO"
  }

  def maxNota(a: Int, b: Int): Int = {

```

```

    if (a > b) a else b
  }

def clasificacion(nota: Int): String = {
  if (nota >= 9) "EXCELENTE"
  else if (nota >= 7) "NOTABLE"
  else if (nota >= 5) "APROBADO"
  else "SUSPENSO"
}

val estudiantes = List("Ana", "Luis", "Marta", "Pedro", "Sofia")

def analizarEvaluacion(nombre: String, notas: Array[Int]): Unit = {
  println(s"--- $nombre ---")

  var i = 0
  var aprobados = 0
  var suspensos = 0
  var mejorNota = -1

  while (i < notas.length) {
    val estudiante = estudiantes(i)
    val nota = notas(i)
    println(s"$estudiante -> $nota -> ${estadoNota(nota)}")

    if (aprobado(nota)) aprobados = aprobados + 1 else suspensos = suspensos + 1
    mejorNota = maxNota(mejorNota, nota)
    i = i + 1
  }

  println("--- Resumen del grupo ---")
  println(s"Estudiantes: ${notas.length}")
  println(s"Aprobados: $aprobados")
  println(s"Suspensos: $suspensos")
  println(s"Mejor nota: $mejorNota")

  println("--- Clasificación ---")
  i = 0
  while (i < notas.length) {
    println(s"${estudiantes(i)} -> ${notas(i)} -> ${clasificacion(notas(i))}")
    i = i + 1
  }
}

val notas = Array(8, 4, 10, 6, 3)
val notasSegundaEvaluacion = Array(9, 5, 8, 7, 6)

analizarEvaluacion("Primera evaluación", notas)
analizarEvaluacion("Segunda evaluación", notasSegundaEvaluacion)

def contarAprobados(notas: Array[Int]): Int = {
  var aprobados = 0
  notas.foreach(n => if (aprobado(n)) aprobados = aprobados + 1)
  aprobados
}

```

```

val aprobados1 = contarAprobados(notas)
val aprobados2 = contarAprobados(notasSegundaEvaluacion)

println("--- Comparación de evaluaciones ---")
println(s"Mejor nota primera evaluación: ${notas.max}")
println(s"Mejor nota segunda evaluación: ${notasSegundaEvaluacion.max}")
println(s"Aprobados primera evaluación: $aprobados1")
println(s"Aprobados segunda evaluación: $aprobados2")

if (aprobados2 > aprobados1) println("El grupo ha mejorado")
else if (aprobados2 < aprobados1) println("El grupo ha empeorado")
else println("El grupo se ha mantenido igual")

val nuevosEstudiantes = "Carlos" :: estudiantes
println(s"Lista original: $estudiantes")
println(s"Lista nueva: $nuevosEstudiantes")
}

```

Salida esperada (resumen):

```

--- Primera evaluación ---
Ana -> 8 -> APROBADO
Luis -> 4 -> SUSPENSO
Marta -> 10 -> APROBADO
Pedro -> 6 -> APROBADO
Sofia -> 3 -> SUSPENSO
--- Resumen del grupo ---
Estudiantes: 5
Aprobados: 3
Suspendidos: 2
Mejor nota: 10
...
--- Comparación de evaluaciones ---
Mejor nota primera evaluación: 10
Mejor nota segunda evaluación: 9
Aprobados primera evaluación: 3
Aprobados segunda evaluación: 5
El grupo ha mejorado
Lista original: List(Ana, Luis, Marta, Pedro, Sofia)
Lista nueva: List(Carlos, Ana, Luis, Marta, Pedro, Sofia)

```

Documentación: nuevosEstudiantes se crea con ::, que construye una lista nueva con "Carlos" como cabeza y reutiliza estudiantes como cola; por eso la lista original (estudiantes) permanece intacta y ambas coexisten en el programa.